

IVIVC App User Guide

Contents

1. What the app does	1
2. Input datasets	1
3. Analysis approaches and models	2
4. Preprocessing options	2
Normalization	2
Scaling	3
Interpolation	3
5. Cross-validation options	3
6. Grid-search initialization	4
7. Performance metrics	5
8. Uncertainty and error bars	6
Approach 1: Direct Correlation	6
Approach 2: Independent Model Fitting	7
Approach 3: Direct Mapping	7
9. Outputs	7
10. Command-line interface	8
11. Regenerating this guide	8

Establishing an in vitro-in vivo correlation (IVIVC) can support the evaluation of medical devices that include absorbable polymers by relating in vivo changes in material properties over time to corresponding in vitro test results, such as degradation profiles. IVIVCs may help evaluate manufacturing or material changes, support predictions of in vivo performance when modifying an existing device design, and reduce reliance on animal studies during preclinical development. This tool is intended to facilitate IVIVC development. The user supplies paired in vitro and in vivo fitting data, and may optionally supply an in vitro prediction dataset. The app applies selected preprocessing steps, fits and compares multiple correlation approaches and model families, evaluates selected performance metrics and cross-validation results, generates fitting and prediction plots, and produces report-ready summaries that can be reviewed and incorporated into external documentation.

1. What the app does

- Apply selected normalization, scaling, and interpolation options to prepare fitting and prediction data.
- Fit user-selected IVIVC approaches and model families to paired in vitro and in vivo data.
- Calculate selected goodness-of-fit, model-comparison, and cross-validation metrics.
- Generate fitting, comparison, and prediction plots with uncertainty displays where supported.
- Create report-ready HTML and editable Word summaries for user-selected final models or methods.

2. Input datasets

Input	Description
Fitting dataset	Required four-column dataset used to establish the in vitro/in vivo relationship. Columns are interpreted as in vitro time, in vitro value, in vivo time, and in vivo value.

Input	Description
Prediction dataset	Optional two-column in vitro dataset used after fitting to generate in vivo predictions. Columns are interpreted as in vitro time and in vitro value.

For Excel workbooks, select the sheet containing the relevant data. For CSV files, the first columns are read directly in the order described above.

3. Analysis approaches and models

Approach	Description	Available models
Approach 1: Direct Correlation	Approach 1 fits a direct relationship between matched in vitro and in vivo time points after preprocessing. The fitted model is then used as a time-scaling function that maps an in vitro time to a predicted in vivo time.	Double Exponential, Exponential, Linear, Linear (zero intercept), Power
Approach 2: Independent Model Fitting	Approach 2 fits the same model family independently to the in vitro and in vivo datasets. With fitting data alone, the report summarizes the two fitted models and their fitted-curve uncertainty. When prediction data are supplied, predictions can be generated by comparing the fitted in vivo and in vitro responses or by comparing fitted characteristic time constants when those values are available.	Double Exponential, Exponential, Mass Loss (Gompertz), Mass Loss (One-Phase), Mass Loss (Surface Erosion), Mass Loss (Two-Phase), Power, Stretched Exponential
Approach 3: Direct Mapping	Approach 3 uses direct interpolation-based mappings between datasets rather than fitting a parametric model. It can describe value-ratio behavior over time and time-ratio behavior over value ranges. When prediction data are supplied, those mappings can be used to scale prediction values or prediction times.	No parametric model selection

4. Preprocessing options

Normalization

Option	Internal key	Description
Min-max normalization	<code>min_max</code>	Scales response values to the 0-1 range using the observed minimum and maximum within each dataset.
Initial-value normalization	<code>initial_value</code>	Divides response values by the first response value in the corresponding dataset.

Scaling

Option	Internal key	Description
Log10 time scaling	<code>log_x</code>	Applies a base-10 logarithm to time values before fitting or interpolation.
Log10 response scaling	<code>log_y</code>	Applies a base-10 logarithm to response values before fitting or interpolation.

Interpolation

Option	Internal key	Description
No interpolation	<code>none</code>	Uses only exact raw alignments. No values are estimated between observations; unavailable aligned positions are marked as missing.
Shared-time interpolation	<code>default</code>	Interpolates both datasets onto the common set of experimental time points, using linear interpolation and marking out-of-range values as unavailable.
Densified interpolation	<code>alternative</code>	Creates additional linearly spaced points between adjacent observations before interpolation. This can smooth displays and make fitting more stable, but adds more interpolation-derived points that skew statistical comparisons.

5. Cross-validation options

Cross-validation provides an approximate check of how a fitted model performs on held-out data. It is most useful when there are enough observations to fit the model repeatedly while leaving some observations out for evaluation.

The available schemes are Shuffle split, K-fold, Leave-one-out, and None. Shuffle split uses repeated random holdout sets, K-fold uses systematic fold-based holdouts, Leave-one-out holds out one observation at a time, and None disables cross-validation. For small IVIVC datasets, cross-validation summaries should be interpreted as exploratory diagnostics rather than definitive validation results.

Scheme	Internal key	Description
Shuffle split	<code>shuffle_split</code>	Repeatedly creates random training and holdout subsets. This can be useful when the dataset has enough observations to support repeated resampling. The test-size parameter controls the fraction held out in each split, and the random seed makes the random splits reproducible.

Scheme	Internal key	Description
K-fold	<code>kfold</code>	Divides the dataset into a specified number of folds, fits the model on all but one fold, and evaluates on the held-out fold. Each fold is used once as the holdout set. The <code>splits/folds</code> parameter controls the number of folds.
Leave-one-out	<code>leave_one_out</code>	Uses one observation as the holdout set and fits the model on all remaining observations, repeating until each observation has been held out once. This can be useful for very small datasets, but the results can be variable and computationally slower.
None	<code>none</code>	Disables cross-validation. This may be appropriate when the dataset is too small for meaningful holdout validation or when only fitted-model summaries are needed.

Parameter	Internal key	Description
Cross-validation scheme	<code>cv_scheme</code>	Selects the splitting method used to estimate held-out performance for fitted parametric models.
CV splits/folds	<code>cv_n_splits</code>	For Shuffle split, the number of repeated random train/test splits. For K-fold, the number of folds. This setting is not used for Leave-one-out or None.
ShuffleSplit test size	<code>cv_test_size</code>	The fraction of observations held out for testing in each Shuffle split repeat. Smaller values leave more observations for fitting; larger values make the holdout test more demanding but leave fewer observations for training.
CV random seed	<code>cv_random_state</code>	Seed used to make randomized cross-validation splits reproducible. This applies to Shuffle split and shuffled K-fold.

6. Grid-search initialization

The app conducts an automated grid search to identify suitable starting parameter values for each model before nonlinear least-squares fitting, reducing the need for manual parameter tuning. Candidate values are sampled within the configured real-space parameter bounds and are used only to initialize the final curve fit.

This initialization step can improve convergence and reduce sensitivity to poor starting values. It is not itself the final fitting algorithm and does not guarantee that the final fitted parameters are the global optimum.

Parameter	Internal key	Description
Grid-search random starts	<code>grid_search_num_points</code>	Number of candidate starting parameter sets evaluated before nonlinear least-squares fitting. Higher values may improve robustness but increase runtime.
Grid-search CPU cores	<code>grid_search_num_cores</code>	Number of CPU cores used for the starting-point search. Blank or Auto uses the app default.
Grid-search parameter minimum	<code>grid_search_param_min</code>	Lower real-space bound for randomly sampled candidate starting parameter values.
Grid-search parameter maximum	<code>grid_search_param_max</code>	Upper real-space bound for randomly sampled candidate starting parameter values.
Grid-search random seed	<code>grid_search_random_state</code>	Seed used to make randomized starting-point sampling reproducible. Changing the seed changes the sampled candidates while keeping the same bounds and number of starts.

7. Performance metrics

Metric	Internal key	Preferred direction	Description
Adjusted R^2	<code>adjusted_r_squared</code>	Higher is generally better	R^2 adjusted for the number of fitted parameters. Higher values are better and can penalize unnecessary model complexity.
R^2	<code>r_squared</code>	Higher is generally better	Coefficient of determination. Higher values indicate that the model explains more variance in the observed response.
MSE	<code>mse</code>	Lower is generally better	Mean squared error between observed and fitted values. Lower values indicate smaller average squared residuals.
RMSE	<code>rmse</code>	Lower is generally better	Root mean squared error, in the same response units as the fitted data. Lower values indicate smaller typical residuals.
NRMSE (mean)	<code>mnrms</code>	Lower is generally better	RMSE normalized by the response mean. Lower values indicate smaller error relative to the response magnitude.

Metric	Internal key	Preferred direction	Description
NRMSE (std. dev.)	<code>nrmse</code>	Lower is generally better	RMSE normalized by the response standard deviation. Lower values indicate smaller error relative to response variability.
AIC	<code>aic</code>	Lower is generally better	Akaike Information Criterion. Lower values indicate a better fit-complexity tradeoff. AIC is intended for comparison of models fitted to the same dataset, not as an absolute goodness-of-fit measure.
BIC	<code>bic</code>	Lower is generally better	Bayesian Information Criterion. Lower values indicate a better fit-complexity tradeoff, with a stronger penalty for additional parameters than AIC. BIC is intended for comparison of models fitted to the same dataset, not as an absolute goodness-of-fit measure.

8. Uncertainty and error bars

The following descriptions summarize how fitted-parameter uncertainty, fitting-plot bands, and prediction error bars are calculated or interpreted for each approach.

Approach 1: Direct Correlation

Fitting uncertainty

Parameter estimates: fitted parameters are reported as estimate $\pm$ standard error. Standard errors are derived from the nonlinear least-squares parameter covariance matrix. Correlations among fitted parameters are retained when uncertainty-aware parameter values are created.

Fitting plot bands: shaded 95% prediction bands are generated by Monte Carlo sampling of the fitted parameter covariance matrix. The sampled model variance is combined with the residual variance from the fitted data, then converted to a two-sided interval using a t multiplier when residual degrees of freedom are available.

These bands describe approximate prediction uncertainty around the fitted relationship, not regulatory acceptance limits. They depend on the fitted model form, local covariance approximation, residual behavior, and preprocessing choices.

Prediction uncertainty

Prediction plot error bars: when a prediction dataset is supplied, predicted in vivo times are plotted with horizontal error bars. The bars are calculated from the same 95% prediction-band calculation used for the fitted correlation model; the upper and lower bar lengths are the distances from the band mean to the corresponding upper and lower interval bounds at each input prediction time.

The error bars therefore include propagated fitted-parameter uncertainty and residual/model error from the Approach 1 correlation fit.

Approach 2: Independent Model Fitting

Fitting uncertainty

Parameter estimates: each in vitro and in vivo model parameter is reported as estimate $\pm$ standard error. Standard errors are derived from the corresponding nonlinear least-squares covariance matrix, with fitted-parameter correlations retained in uncertainty-aware values.

Time constants: tau values, when available, are calculated from the fitted model and parameter covariance. The reported tau uncertainty is the propagated standard uncertainty from that covariance-based calculation.

Fitting plot bands: the in vitro and in vivo shaded 95% prediction bands are generated separately. For each fitted model, the app Monte Carlo samples the fitted parameter covariance matrix, combines sampled model variance with residual variance, and converts the resulting standard uncertainty to a two-sided interval using a t multiplier when residual degrees of freedom are available.

Prediction uncertainty

Value-ratio prediction error bars: uncertainty is propagated from both fitted models. The app samples each model's parameter covariance matrix, calculates the in vivo/in vitro response ratio for each sample, combines ratio variance with residual-based ratio uncertainty, and converts that uncertainty to a two-sided 95% interval. The plotted vertical error bars are scaled from the resulting ratio interval at each prediction time.

Time-constant-ratio prediction error bars: when both tau values are valid, uncertainty in each tau is propagated through the tau ratio using uncertainty-aware arithmetic. The standard uncertainty of the scaled prediction time is multiplied by a t value, where available, and plotted as a symmetric horizontal error bar.

Ratio-based uncertainty can become unstable when the denominator model prediction or denominator time constant is near zero. Predictions in those regions should be interpreted cautiously.

Approach 3: Direct Mapping

Fitting uncertainty

Parameter uncertainty: Approach 3 does not fit a parametric model, so there is no fitted-parameter covariance matrix, no parameter standard errors, and no model-based 95% prediction band.

Fitting plots: the plots show interpolation-based ratios/mappings calculated from the preprocessed datasets. They should be interpreted as deterministic mappings from the supplied data and preprocessing choices, not as statistical uncertainty bands.

Prediction uncertainty

Prediction plot error bars: Approach 3 currently does not generate formal propagated error bars for prediction outputs because no parametric covariance model is fitted.

Uncertainty is therefore limited to the quality of the input data, preprocessing, interpolation, and the assumption that the observed mapping remains valid for the prediction dataset. Extrapolated points outside the interpolation range are excluded or returned as unavailable.

9. Outputs

Output	Description
Preprocessing outputs	Processed data tables and plots showing the data used for fitting and prediction.
Model fitting outputs	Parameter estimates, parameter standard errors where available, fitted plots, and selected performance metrics.
Model comparison outputs	Tables comparing selected approaches, models, metrics, and cross-validation summaries.
Prediction outputs	Prediction plots and downloadable tables when a prediction dataset is supplied and the selected method can be applied.

Output	Description
Selected model & summary of outputs	User-selected report summaries and optional Word report export for incorporation into external documentation.

Output availability depends on the selected approaches, whether fitting succeeds, and whether a prediction dataset was supplied.

10. Command-line interface

The repository includes `cli.py` for batch-oriented fitting from the command line. Use `python cli.py --help` to see available arguments for input files, approaches, models, metrics, preprocessing options, cross-validation settings, and grid-search settings. The internal keys specified in this user guide are used as settings with `cli.py`.

11. Regenerating this guide

Most descriptive text in this guide is generated from `utilities/descriptions.py`, with routine wording overrides in `utilities/descriptions.yaml`. After editing those files, regenerate the Markdown files with:

```
python utilities/generate_user_guide.py
```

To generate a PDF, use a Markdown-to-PDF tool such as Pandoc:

```
pandoc docs/user_guide.md -o docs/user_guide.pdf
```